

31. Online Auction (eBay-style bidding)

Interviewer: Design an online auction platform, eBay-style. A seller lists an item with an end time. Buyers place bids; each new bid must be strictly higher than the current highest. Anyone watching the item sees the price update **live**. When the clock hits zero, we need exactly one winner, and that winner pays. Go.

Candidate: This is a fun one because it looks like a simple CRUD app on the surface — "store bids, show the highest" — but it's actually a **concurrency and real-time correctness** problem wearing a CRUD costume. Let me clarify before I design.

1. Clarifying the requirements

Candidate: A few questions: 1. "Strictly higher" — is there a minimum increment (e.g. must beat current price by at least \$1), or is \$0.01 higher valid? 2. How many people typically watch/bid on one item — dozens, or could a hot item (a PS5, a celebrity-owned guitar) draw tens of thousands of live watchers? 3. Do we need **live price push** to watchers, or is polling acceptable? 4. At close time — is there a hard, fair cutoff, or do we need to defend against **last-millisecond sniping**? 5. Is there a payment step on win, and what happens if the winner's payment fails?

Interviewer: Minimum increment, yes — configurable per item, say \$1 or 5%. A hot item can have **100,000 concurrent watchers**. Live push, not polling — nobody should have to refresh. Sniping is a real concern; assume bidders will try to bid in the literal last millisecond. Yes, payment on win; if it fails, we should have a sane fallback, not just lose the sale.

Candidate: Good, that sharpens it. Requirements:

Functional - `POST /bid` → accepted only if `amount >= current_price + min_increment`; returns `BID_ACCEPTED(new_price)` or `BID_REJECTED(current_price)` or `AUCTION_CLOSED`. - Watchers get **real-time price updates** (no polling) as bids land. - At `end_time`, the system determines **exactly one winner** — deterministically, no matter how many bids land in the final moments. - Winner is charged; on payment failure, fall back sanely (not just "sale lost"). - Full, auditable **bid history** per item.

Non-functional — where it's interesting - Correctness over throughput. Unlike a giveaway or flash sale, the write rate on any *single* item is naturally bounded by human bidding speed (bots aside) — this is not a 670k/s hot-key problem. The hard part is **exactness**: never accept a bid that isn't strictly higher than the current price at the instant it's evaluated, and never let two bids both "win" the same race. -

Real-time fan-out at scale: one hot item, 100k watchers, sub-second price propagation. -

Deterministic close: the winner at `end_time` must be unambiguous, immune to network jitter and client clocks. - **Money correctness:** charge exactly the winner, exactly once.

2. Estimation

Platform-wide: assume 5M active auctions, average a few bids/hour each -> the AGGREGATE write rate across all items is modest (a few thousand bids/sec platform-wide).

The number that actually matters is PER-ITEM concentration:

a hot item with 100,000 watchers might see a bidding flurry in the last 30s of its close -> maybe a few hundred bid ATTEMPTS/sec on that ONE item, not millions. Contrast with the burger giveaway's 670k/s on one counter -- auctions are nowhere near that regime. This changes the design: we don't need to shard a hot counter across 100 keys; a single, atomically-updated row is plenty fast for the write side.

The real scale problem is the READ / PUSH side:

100,000 watchers on one item, price updates a few times/sec during the flurry -> up to $\sim 100,000 \times \text{a few} = \text{hundreds of thousands of PUSH events/sec}$ fanned out from one source. That's a fan-out problem, not a write-contention problem.

Bid history: 5M auctions * ~ 50 bids avg = 250M rows/year. Trivial for a relational DB with proper indexing.

Candidate: So two distinct engineering problems, not one: **(a)** an atomic, low-contention write path for accepting a bid, and **(b)** a **fan-out** problem for pushing price updates to a huge, item-concentrated audience. I'll design both.

3. A simple V1 (and where it breaks)

Candidate: Let me state the naive version, because it exposes the exact bug the interviewer is fishing for.

```
POST /bid {itemId, amount}
|
▼
BEGIN;
  SELECT MAX(amount) FROM bids WHERE item_id = :id;  -- "what's the current high?"
  -- app code compares: is :amount > current_max?
  if yes: INSERT INTO bids(item_id, bidder_id, amount, ts) VALUES (...);
COMMIT;
```

Why this is dangerous: the check ("what's the max?") and the act ("insert my bid") are **two separate steps with a gap between them**. Two bids can both read the *same* current max before either has inserted, both independently conclude "I'm higher, I'm good," and **both get inserted as accepted** — even if they're for the identical amount, or even if the second one is actually *lower* than a bid that landed in the gap. This is the classic **check-then-act race**, and it's exactly what happens under real concurrency at the closing seconds of a hot auction. There's also a second problem: computing `MAX(amount)` (or `ORDER BY amount DESC LIMIT 1`) gets slower as bid history grows, and doing this exact lazy-computation trick **at close time**, under the heaviest concurrency of the whole auction, is the worst possible moment for a race to bite.

4. The bid write path — an atomic conditional update

Interviewer: Right, that's the bug. How do you actually accept a bid safely?

Candidate: The fix is the same shape as any check-then-act race: **collapse the check and the act into one atomic statement**, so there is no gap for a second bid to land in. I keep an **authoritative "current highest" pointer** as a column on the auction row itself, and update it with a single conditional `UPDATE` — a compare-and-set (CAS) done natively by the database:

```
UPDATE auctions
SET    current_price      = :amount,
       current_bidder_id = :bidder_id,
       current_bid_id    = :bid_id,
       version            = version + 1
WHERE  id = :item_id
AND    status = 'ACTIVE'
AND    :amount >= current_price + min_increment
RETURNING version;
```

- If it returns **1 row** → the bid is accepted; this statement *is* the commit point.
- If it returns **0 rows** → either the auction is closed, or — critically — **someone else's bid already raised the price past this one** between when the user typed their bid and when it arrived. Reject with the *current* price so the client can immediately try again higher.

Why this closes the race: two concurrent `UPDATE` s against the *same row* are serialized by the database's own row-level locking (MVCC) — there is no window where both can read the old price and both succeed. Whichever commits first wins; the second one's `WHERE` clause simply no longer matches, atomically, in the same statement that would have granted it. No app-level "read, compare, write" gap exists at all.

I pair the price-pointer update with an **append-only insert into a `bids` table in the same transaction**, so I get both: a fast, race-free "what's the current price" pointer, and a full, immutable audit trail of every bid (accepted or not) for history and disputes.

```
BEGIN;
  UPDATE auctions SET current_price=..., ... WHERE ... AND :amount >= current_price + inc;
  -- 0 rows? ROLLBACK, return BID_REJECTED(current_price)
  -- 1 row? INSERT INTO bids(item_id, bidder_id, amount, accepted=true, ts) ...;
COMMIT;
```

Note this single-row update is **not** the burger-giveaway hot-lock problem: that failure mode was 670,000 requests/sec on *one* counter. A single auction sees at most a few hundred bids/sec even at its hottest — a plain indexed `UPDATE` on one row handles that with room to spare. No sharding needed on the write side; the real scaling problem is next.

5. Real-time price push — the fan-out problem

Interviewer: Good, the write is safe. Now — 100,000 people are watching this item. How do they see the price change *without* polling?

Candidate: This is a pure fan-out problem: **one write, many readers, low latency.**

```
[Bidder]
  | POST /bid → atomic UPDATE commits ($4)
  ↓
[Auction service]
  | after commit: publish {itemId, newPrice, bidderMasked, ts}
  ↓
[ Redis Pub/Sub channel: "auction:{itemId}" ]
  |
  ├──→ [WS gateway pod 1] — holds ~5k sockets — pushes to its local subscribers
  ├──→ [WS gateway pod 2] — holds ~5k sockets — pushes to its local subscribers
  ├──→ ...
  └──→ [WS gateway pod N] — holds ~5k sockets — pushes to its local subscribers
```

100,000 watchers / ~5,000 sockets-per-pod ≈ 20 gateway pods subscribed to ONE channel.

- Watchers hold a **WebSocket** connection to a stateless-ish gateway fleet, subscribed to the item's channel when they open the listing.
- **Redis only fans out to the ~20 gateway pods**, not to 100,000 individual sockets — each gateway does the last-mile fan-out **in-process** to its own connected clients. That keeps the fan-out cost on Redis tiny (tens of subscribers per channel) no matter how many browsers are watching.
- **Why WebSocket and not polling?** 100k clients polling even once every 2s is 50k reqs/sec of read load on one item for a number that mostly isn't changing — wasteful and still laggy. Push delivers updates in milliseconds and costs nothing when the price is quiet.
- **Why Redis Pub/Sub and not each gateway polling Postgres?** Pub/Sub is literally built for "broadcast this event to whoever's listening right now," at sub-millisecond fan-out to a bounded number of subscribers (gateway pods) — exactly our shape.
- Also cache the **current price** itself in Redis (`SET auction:{id}:price ...` , updated write-through right after the CAS commits) so a plain `GET /auctions/{id}` (for a client that just loaded the page, or a fallback poll) never has to hit Postgres under load.

6. Auction close — determinism, not vibes

Interviewer: The clock hits zero. How do you pick exactly one winner, with bids flying in at the very last moment?

Candidate: The close has to be **just another atomic conditional update on the same row**, so it composes with the bid-accept CAS instead of racing against it in application code:

```

UPDATE auctions
SET    status = 'CLOSED', closed_at = now()
WHERE  id = :item_id
      AND status = 'ACTIVE'
      AND now() >= end_time
RETURNING current_bidder_id, current_price;

```

A background **close worker** (a scheduler that wakes up per item at its `end_time`, not a naive "poll every item every second" cron) fires this. Because both the bid-accept `UPDATE` (§4) and the close `UPDATE` condition on `status = 'ACTIVE'` against the **same row**, Postgres serializes them exactly like any other pair of concurrent writers: whichever one commits first wins, and the other's `WHERE` clause simply fails to match — no ambiguous in-between state, no "was this bid before or after close" argument. I go deeper on the exact millisecond race in Deep-dive B.

Two more deliberate choices: - **Server receive-time is authoritative, never client clock.** A bid is "on time" if it reaches the server and commits before the close `UPDATE` — never based on when the client says it hit "submit." - **Anti-snipe extension.** If a bid is accepted within the last N seconds (say 10s) of `end_time`, extend `end_time` by N seconds **in the same transaction as the bid**. This gives other watchers a fair chance to respond instead of rewarding whoever has the lowest network latency in the final millisecond — a real mitigation for sniping, covered further in Deep-dive B.

7. API + data model

API

POST	/auctions	create a listing (seller, item, start/end, min_increment)
GET	/auctions/{id}	current state (price, status, end_time) — cached read
POST	/auctions/{id}/bids	{amount, idempotencyKey} → BID_ACCEPTED BID_REJECTED C
GET	/auctions/{id}/bids	paginated bid history (audit)
WS	/auctions/{id}/watch	subscribe to live price-update stream

Data model (PostgreSQL)

```

auctions(
  id, seller_id, item_id, status ENUM('ACTIVE','CLOSED','SETTLED'),
  start_time, end_time, min_increment,
  current_price, current_bidder_id, current_bid_price, version,
  INDEX(status, end_time)          -- close worker scans upcoming closes
)

bids(
  id, auction_id, bidder_id, amount, accepted BOOLEAN, created_at,
  INDEX(auction_id, created_at)    -- history queries, append-only, immutable
)

orders(
  id, auction_id, winner_id, amount, payment_status, created_at
)

```

Why relational, not a KV/NoSQL store as the source of truth? The whole design hinges on a **single atomic conditional update plus an append-only insert committing together** — that's a transaction, and I want ACID guarantees on money and on "who won." A NoSQL store with conditional writes (e.g.

DynamoDB's `ConditionExpression`) *could* do the CAS itself, but I'd lose the ability to atomically also write the bid-history row and, later, join auctions to orders to users for settlement and disputes — all things this domain needs constantly. Given the actual write volume is modest (§2), Postgres's stronger consistency story is free; I'm not paying a throughput tax to get it.

8. Payment on win, and async work off the critical path

Interviewer: Winner's decided. Now they pay. What's synchronous, what isn't?

Candidate: Determining the winner **must** be synchronous and strongly consistent — that's the one thing that can never be "eventually correct." Everything downstream of "we know the winner" can be async:

```
[Close worker commits CLOSED, returns winner_id + price]
├── synchronous: create ORDER row (winner, amount, status=PENDING)
└──
    [ Kafka: auction-closed events ]
    ├── [Payment service] – idempotent charge; on success → order.status = PAID
    │   └── on failure → mark order FAILED, re-open a short "second-chance" window
    │       and offer the item to the next-highest bidder (from bid history) –
    │       the classic auction-site fallback, instead of just losing the sale.
    ├── [Notification workers] – "you won!" / "you were outbid" emails/pushes
    └── [Analytics / search indexing]
```

- **Idempotency key on POST /bid** too — a client retry after a network timeout must not double-submit; the server stores (`idempotencyKey` → `result`) and replays the original response.
- Payment failure doesn't need to be catastrophic: because we kept the full bid history, we can deterministically fall back to the next-highest bidder rather than just voiding the sale.

9. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Authoritative price / winner	PostgreSQL, single-row conditional UPDATE (CAS)	ACID correctness on money and "who won" matters more than raw throughput here; per-item write rate is modest (§2), so a plain indexed row update is fast with no sharding needed. <i>Not an in-memory store as the source of truth</i> — a crash/failover losing "who's currently winning" is unacceptable when real money is on the line.
Bid history / audit	Postgres append-only bids table , same transaction as the CAS	Immutable ledger, joinable to orders/users for disputes; committing in the <i>same</i> transaction as the price update means history and current-state can never disagree.
Current-price cache	Redis (string per auction) , write-through after commit	Lets <code>GET /auctions/{id}</code> and reconnecting clients read the price without hitting Postgres under heavy watcher load; it's a cache of the DB, never the source of truth.

Concern	Choice	Why this, and why not the alternative
Real-time fan-out	Redis Pub/Sub → WS gateway fleet	Broadcasts one event to a bounded number of gateway subscribers (tens, not 100k) at sub-ms latency; gateways do the last-mile fan-out in-process. <i>Not polling</i> — 100k clients polling wastes reads and still lags.
WebSocket layer	Dedicated stateless WS gateway fleet , sharded by connection, subscribed per-item	Persistent, bidirectional, push-capable connections at scale; horizontally scales with watcher count independent of the DB.
Async event backbone	Kafka for auction-closed, payment, notification, analytics events	Durable, replayable, ordered per auction — decouples payment/notifications from the close path so a slow downstream never blocks determining the winner.
Payment	Dedicated, idempotent payment service	Isolates money handling; idempotency keys prevent double-charging the winner on retry.
Auction-close scheduling	Durable per-item scheduler (delayed job / timer service), not a blind cron sweep	Fires exactly at <code>end_time</code> per item using the server's authoritative clock; a naive "scan all rows every second" cron doesn't scale and adds close-time jitter.

Say-it-out-loud justification: *"The current price and the winner live in Postgres because this is money and correctness beats throughput — a single conditional `UPDATE` gives me an atomic compare-and-set without ever needing a hot-key workaround, since per-item bid volume is modest. Redis is purely an accelerant: a cache of the current price and the pub/sub backbone that lets a fleet of WebSocket gateways fan a single price change out to 100,000 watchers in milliseconds. Kafka carries everything that doesn't need to block the winner decision — payment, notifications, analytics — off the critical path."*

10. Consistency model — what's strong, what's eventual

- **Strong / linearizable:** `current_price`, `current_bidder_id`, and the close transition — all live on one row, and Postgres's row-level serialization makes the CAS and the close update strictly ordered relative to each other. We never accept a bid below the true current price, and we never produce two winners.
- **Eventual:** what a watcher's browser *displays*. There's a few-millisecond gap between "the price changed in Postgres" and "the pub/sub message reaches your socket." That's fine — it's a display value, not the value used to accept the *next* bid (the next bid is validated fresh against the DB row, not against whatever stale price the client happened to render).
- **Also eventual:** order settlement, notifications, and analytics — all async behind Kafka, as in §8. The one thing that is never allowed to be eventual is "who won."

11. Failure modes & graceful degradation

Failure	What happens	Mitigation
Postgres primary down	Bids can't be accepted	Automated failover to replica; bids fail fast with a clear "try again" rather than hanging; close jobs pause and catch up on recovery
Redis (pub/sub or cache) down	Live push stops; price cache misses	WS gateways fall back to clients re-polling <code>GET /auctions/{id}</code> (hits Postgres directly) — degraded latency, never wrong data, since Postgres is still the source of truth
WS gateway pod crash	Its connected clients drop	Clients auto-reconnect, resync current price via <code>GET</code> , resubscribe to pub/sub — no missed <i>state</i> , only a brief gap in <i>live-ness</i>
Kafka lag	Notifications/payment/analytics delayed	Winner determination is unaffected (doesn't depend on Kafka); consumers catch up; user already sees "you won" from the synchronous path
Payment fails on win	Sale would be lost	Fall back to next-highest bidder from bid history within a short window, instead of voiding the auction
Close-worker instance dies mid-fire	An item's close is delayed	Scheduler is durable (persisted schedule, not in-memory); another worker picks it up; the close <code>UPDATE 's WHERE now() >= end_time</code> guard makes it safe to retry/re-fire

12. Follow-up curveballs

Interviewer: eBay has "proxy bidding" — you set a max, and the system auto-bids for you up to that max as others bid. How does that fit your design?

Candidate: It's the same CAS, just triggered automatically. Store `max_proxy_amount` per bidder per item. Whenever the CAS in §4 succeeds for *someone else's* bid, check if any standing proxy's max exceeds the new price; if so, immediately fire the same conditional `UPDATE` on their behalf (new price = the minimum needed to beat the challenger, capped at their max). It's still one atomic statement per attempt — proxy bidding is a policy layer on top of the same primitive, not a new correctness mechanism.

Interviewer: A client's `POST /bid` times out on the network. Did the bid go through?

Candidate: This is exactly why I put an **idempotency key** on the bid endpoint (§8). The client retries with the *same* key; the server either replays the stored result of the original attempt or, if it never actually landed, processes it fresh. The user never has to wonder or double-bid by retrying.

Interviewer: Seller sets a reserve price and it's not met by close time. What happens?

Candidate: `reserve_price` on the `auctions` row. The close `UPDATE` still fires normally (status → `CLOSED`), but the "determine winner" step checks `current_price >= reserve_price` first; if not met, the order is never created — the item is simply unsold, and the seller is notified. The atomicity guarantee doesn't change; it's an extra condition on what "winning" means.

Interviewer: How do you stop shill bidding — a seller's friend bidding just to drive up the price?

Candidate: Not solvable by the concurrency design, but layered on top: disallow a bid from an account associated with the seller (shared payment method, device fingerprint, IP), rate-limit rapid back-and-forth bidding between the same two accounts, and flag anomalous patterns (one account always "loses" by a tiny margin repeatedly) for review. Orthogonal to the CAS correctness problem, but worth mentioning it's a separate layer.

13. Deep-dive: "Why not just INSERT every bid and compute the max at read/close time?"

Interviewer: You added a `current_price` column and an atomic CAS. That's extra machinery. Why not keep it simple — just `INSERT` every bid into a `bids` table, and compute `MAX(amount)` whenever you need to know the current price, or at close time to find the winner? Isn't a database good at aggregates?

Candidate: It's the natural first instinct, and it's *correct* for a single bid arriving in isolation — the problem is entirely about **concurrency and the moment of evaluation**.

The race: two bids at (or near) the same amount

```
t0  current highest bid on record: $100
t1  Bidder A reads "current max = $100" (via SELECT MAX)
t1  Bidder B reads "current max = $100" (via SELECT MAX)      <- both read the SAME state
t2  A's app logic: "$105 > $100, I'm good" -> INSERT (A, $105)
t2  B's app logic: "$105 > $100, I'm good" -> INSERT (B, $105) <- also passes! same check
t3  Table now has TWO accepted bids at $105. Which one actually wins?
    Neither compared against the OTHER's bid -- only against stale $100.
```

Nothing in the naive design ever atomically links "read the current price" to "commit my bid against exactly that price." Any two requests that overlap in time can both check against the same stale value and both get inserted as "accepted" — even though the rule ("strictly higher than *current*") was violated the instant the second one landed.

The read-time-max latency problem

Even without a tie, computing `MAX(amount)` (or `ORDER BY amount DESC LIMIT 1`) against a growing `bids` table adds latency to *every single bid check*, and that cost is highest exactly when it's most dangerous — during the final concurrent flurry at close time, when many bids are landing and the table is being scanned/locked most heavily by many readers at once. Wrapping the whole "read max, then insert" sequence in a `SERIALIZABLE` transaction would fix the race, but at the cost of transaction

aborts/retries under contention — effectively reinventing a slower, less predictable version of the CAS, instead of just doing the CAS.

Concern	Insert-only, compute max at read/close time	Authoritative current-highest + atomic CAS
Race on equal/near-equal bids	Two bids can both pass a stale check and both get inserted as "accepted"	Impossible — the check and the write are one atomic statement; the second <code>UPDATE</code> simply fails to match
Latency as history grows	<code>MAX / ORDER BY LIMIT 1</code> gets slower with more bids on a popular item	$O(1)$ — a single indexed row lookup/update, independent of history size
Close-time determinism	Computing the winner from history is itself a check-then-act race, at the worst possible moment	The close is just another conditional <code>UPDATE</code> on the same row — composes safely with the bid CAS
Implementation complexity	Looks simpler, but correctness requires <code>SERIALIZABLE</code> isolation + retry logic to actually be safe	One <code>UPDATE ... WHERE ...</code> — the DB does the serialization for free at the row level
Audit trail	Naturally has full history (it's the only table)	Still has full history — the <code>bids</code> insert happens <i>alongside</i> the CAS, not instead of it

Rule of thumb: *Append-only history is great for "what happened," but it's the wrong tool for "is this new thing currently valid" under concurrency. Keep a small, atomically-updated pointer to current state for the check that gates acceptance, and let the history table be purely a record — never re-derive a live decision from a growing log at the exact moment multiple writers are racing to update it.*

14. Deep-dive: the sniping race, and fanning out to 100,000 watchers

Interviewer: Walk me through the absolute worst case: two bids arrive in the **final millisecond** before close, and simultaneously the price has to reach 100,000 watchers live. What actually happens, end to end?

Candidate: Two separate hard problems collide right at close time — a **correctness** race on the row, and a **fan-out** load spike on the push path. Let me take them in order.

The bid race at the exact close boundary

```
end_time = 12:00:00.000

t=11:59:59.997 Bid A arrives at the server: amount=$550
                UPDATE auctions SET current_price=550,... WHERE status='ACTIVE'
                        AND now() < end_time AND 550 >= current_price+inc
                -- Postgres begins processing this UPDATE on the row

t=11:59:59.998 Bid B arrives at the server: amount=$560
                Same UPDATE statement, same row -- BLOCKS behind A's row lock
                (this is normal MVCC row-level serialization, not a bug)

t=11:59:59.999 A's UPDATE commits. current_price is now $550. Row lock released.

t=12:00:00.000 B's UPDATE now proceeds: checks 560 >= 550+inc -> TRUE (if inc<=10)
                B's UPDATE commits. current_price is now $560.

t=12:00:00.000 Close worker's UPDATE fires: status='ACTIVE' AND now()>=end_time
                -- races against B's commit for the SAME row, same serialization rules apply
```

Because the bid-accept `UPDATE` and the close `UPDATE` both condition on `status = 'ACTIVE'` against the **same row**, Postgres guarantees exactly one linear order: either B's bid commits *before* the close flips status (B wins fairly, price is \$560), or the close flips status *first* and B's `UPDATE` then matches zero rows (`status != 'ACTIVE'`) and is cleanly rejected as `AUCTION_CLOSED` — refunding B's attempt with a clear message, never a silent drop or a double-winner. **There is no in-between state** where both look like they succeeded, because row-level locking forces one strict order. This is the entire payoff of using a real CAS instead of app-level check-then-act (Deep-dive A) — the same primitive that fixes the bid race also makes the close race safe, for free.

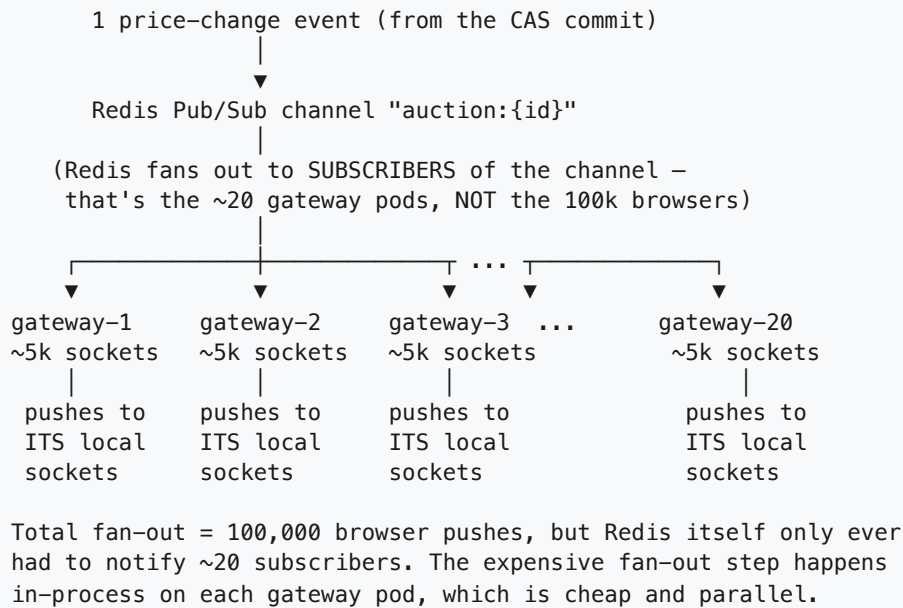
The remaining design question isn't "can two bids both win" (impossible by construction) — it's **"is it fair that whoever has 1ms lower network latency wins the item?"** That's sniping, and it's a fairness problem, not a correctness bug:

Ranked fixes for sniping

1. **Anti-snipe soft-close extension (primary defense).** If an accepted bid lands within the final N seconds (e.g. 10s) of `end_time`, extend `end_time` by N seconds **as part of the same transaction** as the bid CAS. This means a snipe attempt doesn't end the auction — it *reopens* a fair window for others to respond, which is exactly the outcome you want: the highest genuine willingness-to-pay wins, not the lowest ping.
2. **Server receive-time as the only clock that matters.** Never evaluate "on time" using a client-supplied timestamp — only when the `UPDATE` actually commits against the server's `now()`. Removes an entire class of clock-skew/manipulation disputes.
3. **Idempotent, clearly-communicated rejection.** A bid that loses the close race by milliseconds should get an immediate, unambiguous `AUCTION_CLOSED` response (not a timeout or a silent no-op), so the bidder isn't left wondering if their money is at risk.
4. **(Not recommended alone) Just picking a hard, un-extended cutoff.** Simpler, but it directly rewards last-millisecond network advantage over genuine bidding intent — the anti-snipe extension is the difference between "fair auction" and "latency race."

The 100,000-watcher fan-out, at the same moment

The correctness race above happens on **one row**; the fan-out problem is orthogonal and scales with **watchers**, not bidders:



This is why the fan-out doesn't get worse even during the closing-second flurry: the *rate* of price-change events might spike to several per second, but each event's fan-out cost is still bounded by "~20 gateway pods notified via Redis, each pushing to its own in-memory socket list" — it never becomes "Redis pushes directly to 100,000 sockets." Scaling to more watchers just means adding more gateway pods, each subscribing to the same channel; Redis Pub/Sub's cost is a function of subscriber *count* (pods), not message *volume* to end users.

What to say out loud: "The row-level atomic CAS makes the bid race and the close race both linearizable for free — there's never a state where two bids or a bid-and-a-close both 'win' the same row, so sniping is a fairness problem, not a correctness bug. I fix fairness with a soft-close extension keyed off server receive-time, not client clocks. Separately, the 100,000-watcher fan-out is cheap because Redis only ever notifies the gateway fleet — a few dozen subscribers — and each gateway does the expensive last-mile push to its own sockets in-process, so the two hardest-looking problems in this design both collapse to primitives the database and Redis already give you for free."

Summary — the mental model

An online auction is **not** a "scale the hot counter" problem like a giveaway or flash sale — per-item write volume is modest, so the hard part is **exactness under concurrency, not raw throughput**: every bid-accept and the final close must be evaluated with a single **atomic conditional update** on one authoritative row, never a check-then-act read-then-write in application code. That one primitive — a CAS in PostgreSQL — is what makes both the bid race *and* the close race linearizable for free. The second, genuinely hard problem is **fan-out**: pushing a price change to 100,000 live watchers cheaply,

which Redis Pub/Sub plus a WebSocket gateway fleet solves by fanning out to a small number of gateway subscribers and letting each gateway handle its own last-mile push. Payment, notifications, and history are async behind Kafka because they can be — but "who won" never is.